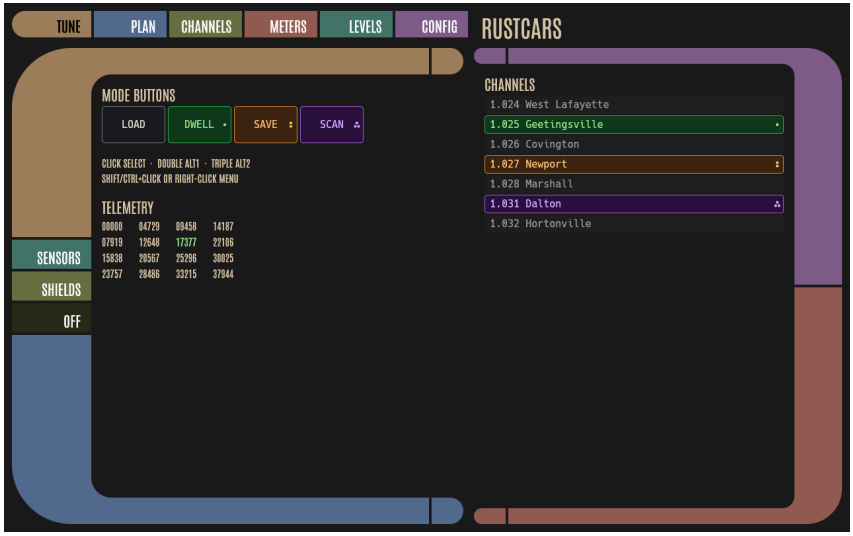
RUSTCARS

DEVELOPER MANUAL

Building LCARS-style interfaces with egui and Rust



DAVID NORRIS · 2026

rustcars Developer Manual. Copyright © 2026 David Norris. The rustcars source code is released under the MIT License (see `LICENSE`).

The bundled typeface **Antonio** is Copyright © 2013 The Antonio Project Authors (github.com/googlefonts/antonioFont), licensed under the SIL Open Font License 1.1. The license text is in `assets/fonts/OFL.txt`; it must be distributed with the font.

LCARS design guidance is adapted from Bracer Jack's *LCARS GuideLine* and *The LCARS Manifesto*, republished at lcars-terminal.de. The structure of this manual is inspired by Apple's *Macintosh Human Interface Guidelines* (1987/1992). It is inspiration only: no Apple text is reproduced, and Apple does not endorse this work. The paint-behind frame technique was inspired by paxel/dedup-rs (MIT).

Star Trek and LCARS are trademarks and/or copyrights of their respective owners (Paramount/CBS). LCARS was designed by Michael Okuda. rustcars is an unofficial fan project and is not affiliated with or endorsed by those owners.

Typeset with Typst. Body: Noto Serif. Headings: Antonio. Code: DejaVu Sans Mono.

CONTENTS

1. Design Philosophy and Principles 1
   1.1 Metaphor: the frame is the instrument panel 1
   1.2 Direct manipulation, see-and-point 1
   1.3 Consistency 2
   1.4 User control 2
   1.5 Feedback and dialog 2
   1.6 Forgiveness 2
   1.7 Perceived stability: controls never move 3
   1.8 Aesthetic integrity 3
   1.9 Modelessness versus click modes 3
2. Getting Started 5
   2.1 Add the crate 5
   2.2 Fonts and license obligations 5
   2.3 Apply the theme 5
   2.4 A minimal application 6
3. Layout and Frames 9
   3.1 Anatomy 9
   3.2 Elbows 9
   3.3 LcarsFrame 10
   3.4 Spacing grid 10
   3.5 Screen composition 10
   3.6 One task per screen 11
4. Controls 13
   4.1 Buttons and click modes 13
   4.2 Lists 14
   4.3 Readouts 14
   4.4 Tabs and navigation segments 14
   4.5 Disabled state 14
   4.6 Focus 15
5. Color 17
   5.1 Palette roles 17
   5.2 State colors 17
   5.3 How many colors 18
   5.4 Contrast rules (WCAG 2.2 AA) 18
   5.5 Verifying 18
6. Typography and Language 19
   6.1 One font, three sizes, two colors 19

6.2 Uppercase labels 19
6.3 Writing labels 19
6.4 Numbers and units 20
7. Feedback, Motion and Sound 21
7.1 Transitions 21
7.2 Blinking and alerts 21
7.3 Progress 21
7.4 Decorative motion 22
7.5 Sound 22
8. Keyboard, Accessibility and Screen Readers 23
8.1 Keyboard 23
8.2 Context menu 23
8.3 Screen readers (AccessKit) 23
8.4 Never color alone 24
8.5 Hit sizes 24
9. Desktop and the Optional Web Interface 25
9.1 Build modes 25
9.2 Where devices live 25
9.3 Security 25
9.4 Portability rules 26
10. Testing 27
10.1 Snapshot tests (kittest) 27
10.2 Contrast tests 27
10.3 Behavior tests 27
10.4 Before you commit 28
11. Example Applications 29
11.1 Scientific calculator 29
11.2 Diary 30
11.3 Ham RF calculators 32
11.4 Timers 34
11.5 Hex viewer 35
11.6 Smith chart 36
11.7 Patterns worth copying 38
11.8 Screenshots 38
12. Checklist and Glossary 41
12.1 Do 41
12.2 Don't 41
12.3 Glossary 41
12.4 Credits 42

1. Design Philosophy and Principles

rustcars brings the LCARS look to ordinary egui applications. LCARS is the fictional computer interface from *Star Trek*, designed by Michael Okuda. Looking like LCARS is only half the job, though. The interface also has to *work* for people doing real tasks. This chapter sets out the principles behind every other chapter. They are adapted from Bracer Jack's *LCARS GuideLine* and *LCARS Manifesto*, and they take inspiration from the structure of classic desktop interface guidelines such as Apple's Macintosh Human Interface Guidelines (1987/1992). The wording and the rules here are our own.

1.1 Metaphor: the frame is the instrument panel

An LCARS screen is a **console**. Colored bars bend around the content like the edge of an instrument panel, and pieces of those bars act as controls. The metaphor tells users two things:

- **Inside the frame** is the information they are working on.
- **On the frame** are the controls that act on it.

Keep that split. Don't scatter controls across the content area when a frame segment could hold them. And don't use frame color merely to label a category; the Manifesto calls a frame that does nothing but label "already a 20% sin".

1.2 Direct manipulation, see-and-point

Users should act on what they can see:

- Every function a screen offers appears as a visible button, list row or segment.

- The user points at it (mouse, touch or keyboard focus) and clicks.
- The mode a control is in is visible right on the control, as its color plus pips.
- Nothing important should be reachable only through a hidden gesture. The context menu (Chapter 8) is a *second* path to the same modes, never the only one.

1.3 Consistency

The same thing looks and behaves the same everywhere:

- **Two spacing constants:** `frame_gap` and `main_gap`.
- **Three font sizes:** title, header, data.
- **Two text colors:** normal and highlight.
- **One set of state colors:** gray, green, orange, purple.
- **One click-mode grammar** for every multi-mode control.

`LcarsTheme` and `palette` hold these values. Never write a color or a pixel size directly in application code.

1.4 User control

The user starts actions; the interface doesn't act on its own initiative.

- Rearranging the screen, starting a scan or changing a mode happens only because the user asked.
- Background activity, such as a scanner stepping through channels, is shown in readouts but never takes over the controls.

1.5 Feedback and dialog

Every action gets an immediate, visible response:

- **On press:** the button shade changes while it is pressed (`tint(base, -0.25)`).
- **On mode change:** the new state color and pip count appear.
- **On data change:** the readout highlights the value being processed (`readout(.., highlight = true)`).
- **When something is slow:** show progress in a readout (Chapter 7) rather than freezing.

1.6 Forgiveness

People make mistakes, so make them cheap:

- Repeating a mode gesture turns the control **off** again (`Mode::after_gesture`).
- Destructive actions ask for confirmation, and where possible offer **undo** instead (see the diary example, Chapter 11).
- Never put a destructive control directly next to a frequently used one.

1.7 Perceived stability: controls never move

The Manifesto is explicit about this. A real interface must not "reconfigure" itself under the user's hands.

- A new task gets **a new screen**.
- New controls may appear, but they must never shift existing ones.
- Transitions are short fades, under a second (Chapter 7).

People build muscle memory for where a control lives. Moving it breaks that memory.

1.8 Aesthetic integrity

Keep the look clean, flat and coherent:

- No gradients, bevels or shadows on the frame.
- At most five meaningful colors.
- Thickness changes at every turn of the frame (the "OMEGA" rule, see the Glossary).
- Decorative animation (scrolling numbers, blinkies) is allowed only when it never gets in the way of the task.

1.9 Modelessness versus click modes

Classic guidelines warn against *modes*: states where the same action does different things. rustcars uses **click modes** on purpose, but keeps them safe:

- **Local:** a mode belongs to one control and one function only. Nothing changes globally.
- **Visible:** the mode is shown on the control by color *and* pips.
- **Few:** at most three modes plus Off; four or more clicks clamp to the third.
- **Reversible:** repeating the gesture turns the control off.
- **Reachable without timing:** Shift/Ctrl modifiers and the context menu reach every mode.

If a function doesn't naturally have related variants, give it a plain single mode. Don't invent alternates just to fill the slots.

2. Getting Started

2.1 Add the crate

```
[dependencies]
rustcars = { git = "https://github.com/webaugur/rustcars" }
egui = "0.37"
eframe = "0.37"
```

The default features are desktop only. The `web` and `web-server` features are opt-in (Chapter 9).

2.2 Fonts and license obligations

`install_lcars_font(ctx)` loads the bundled **Antonio** typeface (`rustcars::ANTONIO_TTF`) as the first proportional font. egui's built-in fonts stay installed as fallbacks.

Antonio is licensed under the **SIL Open Font License 1.1**. If you ship the font, you must:

- ship `assets/fonts/OFL.txt` and the copyright notice from `assets/fonts/FONTS.md` alongside it;
- never sell the font file on its own;
- keep any modified version of the font under the OFL.

To use a different font, call `install_font(ctx, include_bytes!("my.ttf"))`, and check its license before bundling it.

2.3 Apply the theme

```
let theme = rustcars::LcarsTheme::default();
rustcars::install_lcars_font(ctx);
rustcars::apply_theme(ctx, &theme);
```

`apply_theme` does the following:

- forces the dark theme;
- sets the three text styles (Heading, Button, Body/Small/Monospace);
- sets the spacing constants;
- removes shadows and strokes;
- fills panels with `palette::BG`.

Call it once at startup, for example in your `App::new`.

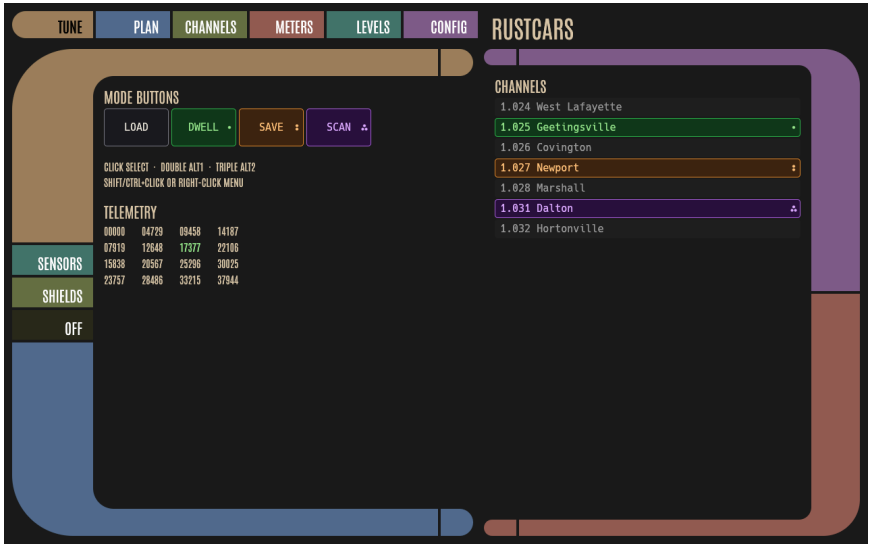
2.4 A minimal application

The complete program is `examples/minimal.rs` (`cargo run --example minimal`). The interesting part:

```
impl eframe::App for App {
    fn ui(&mut self, ui: &mut egui::Ui, _f: &mut eframe::Frame) {
        let t = self.theme;
        let bg = egui::Frame::new().fill(t.bg).inner_margin(t.frame_gap);
        egui::CentralPanel::default().frame(bg).show(ui, |ui| {
            let mut frame = LcarsFrame::new(&t, Side::Left, 1.0);
            frame.min_height = ui.available_height();
            frame.show(ui, &t, |ui| self.content(ui, &t));
        });
    }
}

impl App {
    fn content(&mut self, ui: &mut egui::Ui, t: &LcarsTheme) {
        ui.horizontal(|ui| {
            let seg = egui::vec2(110.0, t.bar_thin);
            lcars_button(ui, t, "Home", t.primary, seg, Cap::Left, true);
            let size = egui::vec2(96.0, 56.0);
            mode_button(ui, t, "Scan", &mut self.scan, size);
        });
        mode_list(ui, t, "channels", &mut self.rows, 28.0);
        let busy = self.scan != Mode::Inactive;
        readout(ui, t, "851.388 MHZ", busy);
    }
}
```

The application owns the state: each `Mode` and the list rows live in your struct. The widgets only read and update them.


TUNE
PLAN
CHANNELS
METERS
LEVELS
CONFIG
RUSTCARS
MODE BUTTONS
LOAD
DWELL
SAVE
SCAN
CLICK SELECT · DOUBLE ALT1 · TRIPLE ALT2
SHIFT/CTRL+CLICK OR RIGHT-CLICK MENU
TELEMETRY
00000 04729 09458 14187
07919 12648 17377 22106
15838 20567 25296 30025
23757 28486 33215 37944
SENSORS
SHIELDS
OFF
CHANNELS
1.024 West Lafayette
1.025 Geetingsville
1.026 Covington
1.027 Newport
1.028 Marshall
1.031 Dalton
1.032 Hortonville

3. Layout and Frames

3.1 Anatomy

An LCARS frame is made of these parts:

- **Vertical bar** (width `bar_thick`, 120 px by default) and **horizontal bar** (height `bar_thin`, 40 px).
- **Elbow (sweep):** the curved joint where the two bars meet. It has an outer radius (about 0.75 × thick) and a concave inner radius (about thin).
- **Cap:** a rounded end that terminates a bar, separated from it by `frame_gap`.
- **Segments:** pieces of a bar cut out with `frame_gap` slits. Segments can act as buttons.

The bars must have **different thicknesses** on either side of a turn. `elbow_shapes` checks this in debug builds.

3.2 Elbows

```
use rustcars::{Corner, Elbow, palette};
let (tan, bg) = (palette::TAN, palette::BG);
let elbow = Elbow::new(Corner::TL, 120.0, 40.0, tan, bg);
elbow.paint(ui, rect); // paint now
// or get the shapes for paint-behind:
let shapes = rustcars::elbow_shapes(rect, &elbow);
```

How the elbow is built:

- **Defined once, mirrored:** the geometry is written once for the top-left corner (TL) and mirrored for TR, BL and BR.
- **Room for the curve:** the swept block is at least `2 × r_out` in size, because egui shrinks a corner radius to half of the rectangle's shorter side.

- **Background must match:** `bg` must be the interior background color, because it is what carves out the concave inner curve.

3.3 LcarsFrame

`LcarsFrame` draws a C-shaped frame (`Side::Left`) or a reversed C (`Side::Right`): top and bottom elbows plus end caps. It is drawn **behind** your content and sized to it:

1. It reserves an empty slot in the paint order with `painter.add(Shape::Noop)`.
2. It lays out your content inside the frame's interior.
3. It fills the slot with `painter.set(...)`, sized to the content's real rectangle.

This technique was inspired by paxel/dedup-rs (MIT).

```
use rustcars::{LcarsFrame, Side};
let mut f = LcarsFrame::new(&t, Side::Right, 0.6); // 0.6 = scale
f.top_color = rustcars::palette::PLUM;
f.bottom_color = rustcars::palette::BRICK;
f.min_height = ui.available_height(); // fill the column
let (_inner, frame_rect) = f.show(ui, &t, |ui| { /* content */ });
```

`frame_rect` is the frame's outer rectangle. Use it to cut segment buttons into the vertical bar, as the demo does.

3.4 Spacing grid

Only two spacing values exist:

- **`frame_gap` (4 px):** between segments, and between bars and caps.
- **`main_gap` (16 px):** between major regions, and between the frame and its content.

Everything else is derived from these. The only exceptions are text and the top row of navigation segments.

3.5 Screen composition

A typical screen:

- **Top:** `egui::Panel::top` holds the tab/navigation strip and the title.
- **Body:** one or two `LcarsFrame`s side by side inside the `CentralPanel`, split on the grid with a `main_gap` between them.
- **Segments:** the left frame's vertical bar holds navigation or mode segments.

Use `ui.scope_builder(UiBuilder::new().max_rect(r), ..)` to place a frame in a computed rectangle. egui's `Margin` type is limited to `i8` (127 px), which is too small for LCARS bars.

3.6 One task per screen

Each screen does one job, such as *Tune*, *Plan* or *Channels*. When the user switches tasks:

- switch the whole screen;
- don't swap controls in place;
- never let new content push existing controls to a new position.

If a control has to appear temporarily, reserve its space in advance or show it in a new screen.

4. Controls

4.1 Buttons and click modes

`mode_button(ui, &t, label, &mut mode, size)` draws a rectangle with a small rounding (4 px), a dark fill tinted by state, a 1 px border and an uppercase monospace label.

Gesture	Keyboard / alternative	Mode	Color	Pips
(none)	context menu: Off	`Inactive`	gray	none
single click	Enter / Space	`Primary`	green	·
double click	Shift+click, Shift+Enter	`Alt1`	orange	:
triple click	Ctrl/Cmd+click, Ctrl/Cmd+Enter	`Alt2`	purple	·:

How a click sequence is resolved (`handle_mode`):

- **Committed once.** egui's `clicked`, `double_clicked` and `triple_clicked` only raise a pending click count. The gesture is applied once the double-click window (0.3 s by default) closes, or immediately on a triple click. So a triple click never also fires the single or double action.
- **Clamped at three.** Four or more clicks count as a triple, and further clicks in the same burst are ignored.
- **Same gesture turns it off.** Repeating the gesture of the current mode switches the control to `Inactive`. A different gesture switches directly to that mode (`Mode::after_gesture`).

When to use each mode

- **Primary (single click):** the ordinary, most frequent meaning, e.g. *Scan*, *Select channel*, *sin*.
- **Alt 1 (double click):** a closely related variant of the same function, e.g. *Scan priority only*, or *asin* (the inverse function).
- **Alt 2 (triple click):** the rarer second variant, e.g. *Scan and record*, or *sinh*.

- **Single-meaning functions:** use `lcars_button` (no modes) or ignore `Alt1`/`Alt2`. Never invent alternates just to fill the slots, and never give the alternates unrelated meanings.
- **Destructive actions:** never put one on a double or triple click, because accidental multi-clicks happen.

4.2 Lists

`mode_list(ui, &t, id_salt, &mut rows, row_h)` takes a `&mut [(String, Mode)]`. Each row behaves exactly like a mode button:

- Inactive rows are flat (`palette::SURFACE`) with muted text.
- Active rows get the state fill, a 1 px border and pips.
- Each row has a screen-reader label like "1.031 Dalton, mode Alt 2".

Keep row heights at 28 px or more on desktop, and 44 px for touch (Chapter 8).

4.3 Readouts

`readout(ui, &t, value, highlight)` shows a data value in the data size. `highlight = true` uses `t.text_hi` to mark the value currently being processed. Put units in the same string and keep their case consistent (Chapter 6).

4.4 Tabs and navigation segments

`lcars_button(ui, &t, label, base, size, cap, enabled)` draws a solid frame segment. It is used for:

- the top tab strip (Tune, Plan, Channels, ...);
- segments cut into a frame bar.

`Cap::Left`, `Cap::Right` and `Cap::Both` round the ends; `Cap::None` gives a plain segment. Label color is chosen automatically with `ink_on`. Segments are for **navigation**, so give them a single meaning and no click modes.

4.5 Disabled state

For segments, pass `enabled = false` to `lcars_button`. It is drawn as `tint(base, -0.6)` and doesn't respond to clicks. WCAG exempts disabled controls from contrast requirements, but the label must still be readable.

Prefer hiding a control that is *never* available on a screen, rather than showing it permanently disabled.

4.6 Focus

Every control responds to the keyboard:

- Tab moves between controls.
- A focused control shows a 1–2 px ring in `t.text`.
- Enter or Space triggers the primary action.

Don't remove the focus ring.

5. Color

5.1 Palette roles

All colors live in `rustcars::palette` and `Mode::colors()`.

Constant	Hex	Role
`BG`	#1B1B1B	Page background, frame interior
`SURFACE`	#202020	List rows, panels
`STRIP`	#303030	Tab strip background
`TAN`	#9C805A	Primary frame color ("Tune")
`SLATE`	#516D8F	Secondary frame color ("Plan")
`OLIVE`	#646F43	Segments ("Channels")
`BRICK`	#945C52	Alert-adjacent frame ("Meters")
`TEAL`	#43736C	Accent ("Levels")
`PLUM`	#815E89	Config / secondary frame
`TEXT`	#D9C7A8	Normal text
`TEXT_MUTED`	#9A9A9A	Inactive list rows

The frame hues come from reference screenshots. They were brightened in the same hue until they reach 3:1 against `BG`.

5.2 State colors

Mode	Fill	Border	Label and pips
Inactive	#1C1C20	#7A7A82	#E8E4D8
Primary	#123A1C	#2EB04E	#96E68C
Alt1	#3D2410	#E08A2E	#FFC27A
Alt2	#2A1040	#A860D6	#DCAAFF

State colors mean *mode*, not *category*. Don't reuse green, orange or purple elsewhere for decoration.

5.3 How many colors

From the guideline:

- **One color** plus its tints and shades is always safe.
- **Three colors** is a good place to stop.
- **A fourth or fifth color** must carry a meaning you can name. In rustcars those are the state colors.
- **Six or more** is to be avoided.

For tints and shades of one hue, use `tint(c, t)` rather than adding new hues.

5.4 Contrast rules (WCAG 2.2 AA)

- **Text: 4.5:1 or more** against its background, at every size we use.
- **Large text (24 px+ regular):** 3:1 would be allowed, but we keep 4.5:1 anyway.
- **Borders, pips, bars and segments (non-text, SC 1.4.11): 3:1 or more** against what's next to them.
- **Never color alone:** modes also show pips; alerts also change their label.

5.5 Verifying

`tests/modes.rs::wcag_contrast` checks every pair and prints a table:

```
cargo test --test modes -- --nocapture
```

For your own colors, use `rustcars::contrast(a, b)` in a test:

```
assert!(rustcars::contrast(my_label, my_fill) >= 4.5);
```

6. Typography and Language

6.1 One font, three sizes, two colors

- **Font:** Antonio, a condensed typeface, used for titles, headers and segment labels. Mode buttons and list rows use egui's monospace font (Hack) so that values line up in columns.
- **Sizes:** `title_size` (36), `header_size` (22) and `data_size` (15). Don't add a fourth size. The only exception is that mode pips scale from `data_size`.
- **Colors:** `t.text` for normal text and `t.text_hi` for highlighted text.

6.2 Uppercase labels

Buttons, segments and section headers are shown in uppercase; the widgets apply `to_uppercase()` for you. Write labels in normal case in your code, e.g. "Scan". That way the screen-reader label stays natural ("Scan, mode Alt 1").

List rows keep their original case, because they are data such as names and frequencies.

6.3 Writing labels

- **Verbs for actions** (SCAN, SAVE, DWELL), **nouns for places** (TUNE, CHANNELS).
- **One or two words.** If a label won't fit, the button is the wrong shape. Make it wider or use a list instead.
- **Don't encode the mode in the label.** The pips show the mode, and the screen-reader label adds the mode name.
- **Same term everywhere** for the same function.

6.4 Numbers and units

- **Fixed decimal places** for values in a column (`851.388`, not `851.39` next to `851.3875`).
- **Leading zeros** for counters that scroll (`format!("{v:05}")`), so their width doesn't jitter.
- **Units** after a space in uppercase (`851.388 MHZ`), or once in the column header.
- **Right-align** numeric columns. The `egui::Grid` in the demo shows the pattern.

7. Feedback, Motion and Sound

7.1 Transitions

When the screen changes:

- **Fade out, then fade in.** Never crossfade.
- **Keep it under one second** in total; about 120 ms out and 150 ms in is plenty. `ctx.animate_bool_with_time(id, visible, 0.15)` gives a 0–1 value to use as opacity.
- **Animation is optional.** Make it switchable for users who prefer reduced motion.

7.2 Blinking and alerts

- **Rate:** blink alerts at about 1 Hz. Never flash between 3 and 50 Hz (photosensitivity, WCAG 2.3.1).
- **More than color:** an alert also changes its label (e.g. "ALERT") or adds an icon.
- **Where alerts appear:** in a fixed place on the frame, never as a pop-up that covers controls.

7.3 Progress

Show long operations in a readout with a value or percentage that the user can read, and highlight it with `text_hi`. Keep the rest of the screen usable.

Refresh at a modest rate, e.g. `ctx.request_repaint_after(Duration::from_millis(250))`, rather than every frame.

7.4 Decorative motion

Scrolling numbers are allowed as decoration, but keep them out of the user's way:

- generate them in code;
- give each element its own period, so they never visibly loop together;
- drive them from `ui.input(|i| i.time)`, never `std::time::Instant`, which panics on wasm.

7.5 Sound

Sound is optional and must never be the only signal. If you use it, keep it short and quiet, and give the user a mute control.

8. Keyboard, Accessibility and Screen Readers

8.1 Keyboard

Key	Effect
Tab / Shift+Tab	Move focus between controls
Enter / Space	Primary (single click)
Shift+Enter, Shift+click	Alt 1, applied immediately
Ctrl/Cmd+Enter, Ctrl/Cmd+click	Alt 2, applied immediately
Right-click / long-press	Context menu listing every mode by name

The modifier shortcuts work for people who can't double- or triple-click reliably. They commit at once, with no timing involved.

8.2 Context menu

Every mode control has a context menu with *Off*, *Select (click)*, *Alt 1 (double-click / Shift)* and *Alt 2 (triple-click / Ctrl)*. The menu doubles as on-screen documentation of the gestures.

8.3 Screen readers (AccessKit)

`mode_button` and `mode_list` rows publish a label of the form *"Scan, mode Alt 1"* through egui's AccessKit integration (`Response::widget_info`). If you build your own controls, do the same:

```
let text = format!("{label}, mode {}", mode.short_name());
let enabled = ui.is_enabled();
resp.widget_info(|| {
    egui::WidgetInfo::labeled(egui::Role::Button, enabled, &text)
});
```

8.4 Never color alone

Every state shows its color **and** a second cue:

- **Modes:** pips (·, ⁚, ·⁚), painted with `paint_pips`. They are drawn shapes rather than font characters, because the triangle character ·⁚ isn't in any of the bundled fonts.
- **Highlighted values:** shown with weight or position as well as color.
- **Alerts:** a changed label as well as color.

8.5 Hit sizes

- **Touch targets:** at least 44 × 44 px; desktop list rows at least 28 px tall.
- **Thin bars:** a frame bar thinner than a finger shouldn't be a button (Manifesto). Place a separate segment there instead.

9. Desktop and the Optional Web Interface

9.1 Build modes

Mode	Command	Notes
Desktop (default)	`cargo run`	Native eframe window, no web dependencies
Browser	`trunk serve` / `trunk build --release`	`--features web`, wasm32, served from `dist/`
Server bridge	`cargo run --features web-server --bin rustcars-web -- --enable`	Serves `dist/` plus a WebSocket at `/ws`

`DemoApp` is the shared `eframe::App` used by both the desktop and web builds. Do the same in your own app: keep one `App` type and a thin, target-specific `main`.

9.2 Where devices live

The web UI runs in the **visitor's browser**. It can't open the server's USB devices or connect to a GQRX TCP port on the server. All device access goes through the server bridge:

- implement `DeviceBackend` (`name`, `handle(cmd) -> reply`) in `src/bin/rustcars-web.rs`;
- the browser sends text commands over `/ws`;
- the shipped `EchoBackend` only answers `ping` with `pong`, so it is a stub.

9.3 Security

- **Off by default:** both features are off, and the server also refuses to start without `--enable`.

- **Localhost first:** it binds `127.0.0.1` by default. `--bind 0.0.0.0:8080` exposes it on the network and prints a warning.
- **No authentication:** the bridge has none. Put an authenticating reverse proxy with TLS, or a VPN, in front of it before exposing it.
- **Static hosting:** GitHub Pages can host the static UI only, with no bridge. Pages for a private repository requires a paid plan.

9.4 Portability rules

- Use `ui.input(|i| i.time)` or the `web-time` crate instead of `std::time::Instant`.
- No blocking I/O in the UI thread. Talk to the bridge asynchronously.
- `apply_theme` forces the dark theme, because browsers otherwise apply the user's light/dark preference.

10. Testing

10.1 Snapshot tests (kittest)

`tests/preview.rs` renders the demo headlessly with `egui_kittest` and wgpu:

```
let mut harness = egui_kittest::Harness::builder()
    .with_size(egui::vec2(1280.0, 800.0))
    .wgpu()
    .build_ui(move |ui| rustcars::demo(ui, &t, &mut st));
harness.run_steps(4);
harness.snapshot("lcars_demo"); // tests/snapshots/lcars_demo.png
```

- **First run:** run `UPDATE_SNAPSHOTS=1 cargo test` to accept a new snapshot.
- **Without a GPU:** install Mesa's software Vulkan driver (lavapipe), e.g. `apt install mesa-vulkan-drivers`.
- **Exact pixels:** snapshots are compared pixel for pixel, so a real GPU may differ slightly from lavapipe. Keep one reference machine, or raise the threshold with `SnapshotOptions`.

10.2 Contrast tests

Add every new color pair to a test that uses `rustcars::contrast` (Chapter 5). The thresholds are 4.5 for text and 3.0 for non-text.

10.3 Behavior tests

The click-mode rules are pure functions that are easy to test:

```
use rustcars::Mode;
// clicks beyond three clamp to Alt2
assert_eq!(Mode::from_clicks(5), Mode::Alt2);
// repeating the current gesture turns the control off
assert_eq!(Mode::Alt1.after_gesture(2), Mode::Inactive);
```

10.4 Before you commit

`scripts/check.sh` runs:

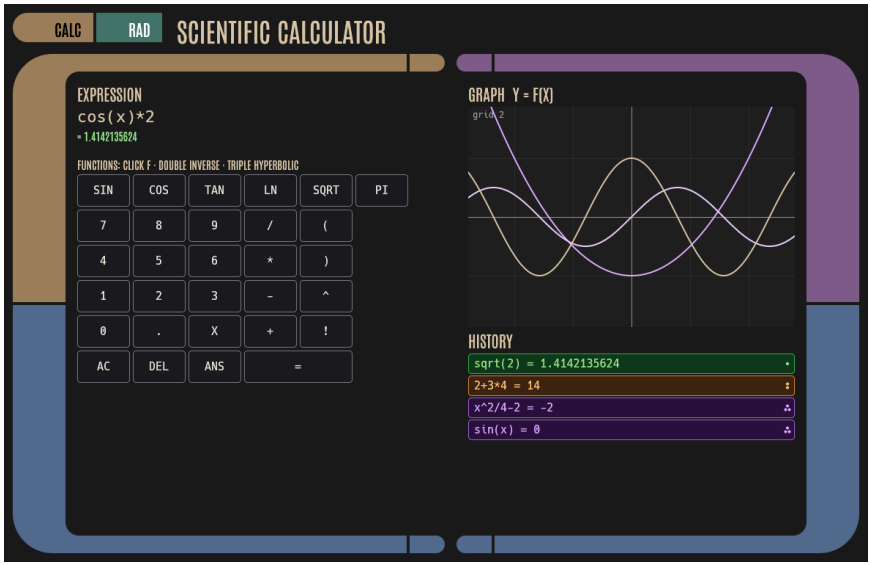
- `cargo fmt --check`;
- clippy (`-D warnings`) for the desktop, wasm and server builds;
- all tests;
- a check that the default build stays free of web dependencies.

11. Example Applications

Six complete desktop applications show the rules in practice. All are built only from rustcars widgets and the theme. They are desktop only; the extra crates they use (`rusqlite`, `directories`, `chrono`) are dev-dependencies, so library users never pull them in.

11.1 Scientific calculator

`cargo run --example calculator`. The source is in `examples/calculator/`: `main.rs` holds the UI and `eval.rs` the expression evaluator.



- **Function keys** are momentary `mode_button`s. The gesture picks the *variant* of one function:

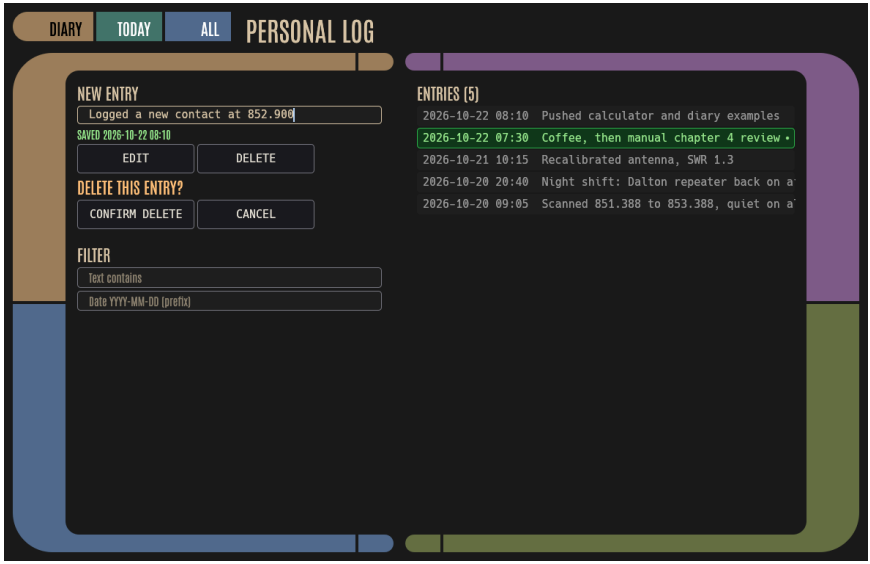
Key	Click (·)	Double (:)	Triple (·:)
SIN, COS, TAN	sin	asin (inverse)	sinh (hyperbolic)
LN	ln	exp (inverse)	log10
SQRT	sqrt	^2 (inverse)	cbrt
PI	π	e	ans

Shift and Ctrl work as Alt 1 and Alt 2, and right-clicking a key lists every variant by name.

- **Digits and operators** use `action_button`, which has no modes, so fast typing never merges into a double click.
- **History rows** (`mode_list`):
  - click inserts the result;
  - double-click loads the expression for editing;
  - triple-click plots it. The row stays purple while plotted; a triple click on it removes the plot.
- **Graph:** painted with egui's painter: a grid with a "nice" step, axes, drag to pan, scroll to zoom. Plotted functions use tints of the Alt 2 purple, all at least 4.39:1 against the plot background. Any input containing `x` is previewed live in the text color.
- **Keyboard:** typing goes straight into the expression; Enter evaluates, Backspace deletes, Escape clears.
- **DEG/RAD** is a navigation segment that toggles the angle unit.
- **Evaluator:** a small recursive-descent parser with precedence, right-associative `^`, factorial, implicit multiplication (`2x`), `pi`, `e`, `ans` and `x`. It has unit tests in `eval.rs`.

11.2 Diary

`cargo run --example diary [-- --db PATH]`. The source is in `examples/diary/`: `main.rs` holds the UI and `store.rs` the SQLite storage.



- **One focused field.** The NEW ENTRY field takes focus whenever no other widget has it, so you can always just type. Enter saves the entry with the current time.
- **Timestamps** are stored as UTC ISO-8601 (`2026-10-22T12:10:00Z`) and shown in local time.
- **Entry rows** use click modes, and only one row is active at a time:
  - click selects the row and shows Edit and Delete buttons;
  - double-click edits the entry in place;
  - triple-click quotes it into a new entry.
- **Forgiveness:** Delete asks for confirmation, and the last delete can be undone with *Undo delete*, which restores the original id and timestamp.
- **Filtering:**
  - text: a case-insensitive substring match where `%` and _ count as literal characters;
  - date: a local `YYYY-MM-DD` prefix, so `2026-10` matches a whole month;
  - the *Today* and *All* segments are shortcuts.
- **Storage:**
  - `rusqlite` with the `bundled` feature (a dev-dependency, so library users never pull it in);
  - schema migrations tracked with `PRAGMA user_version`;
  - the database lives in the platform data directory via `directories::ProjectDirs` unless `--db` is given.

- **Tests:** in-memory database tests cover migration, filtering, editing, delete with undo, timestamps, and reopening a file.

11.3 Ham RF calculators

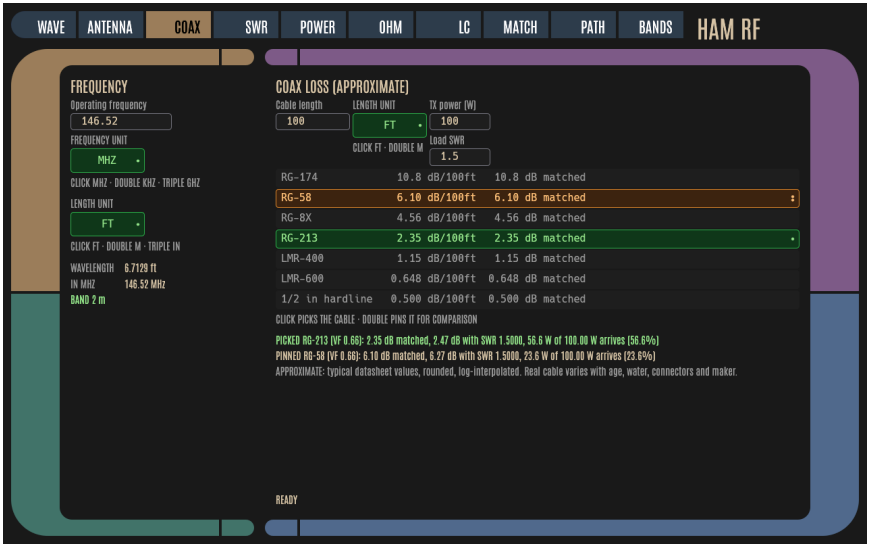
`cargo run --example ham_rf`. The source is in `examples/ham_rf/`: `main.rs` holds the UI, `calc.rs` the RF maths and `bands.rs` the US band table. Both modules are pure and unit tested against known values.



Ten tools sit on the tab strip. One **frequency** field in the left frame feeds every tool that needs it, so you type `14.2` once. Under it are two unit pickers (see 11.7).

Tab	What it does
WAVE	Frequency to wavelength, or wavelength to frequency (a picker chooses the direction). Full, half and quarter wave in m, ft, in and cm.
ANTENNA	Half-wave dipole (468/f ft, with metric), each leg, quarter-wave vertical and radials, full-wave loop, J-pole and 3-element Yagi starting lengths. Rows copy to the clipboard: click = the selected length unit, double = the other system, triple = the whole line.
COAX	Seven common cables (RG-174, RG-58, RG-8X, RG-213, LMR-400, LMR-600, 1/2 in hardline). Loss per 100 ft at the frequency by log-log interpolation, total loss for your length, extra loss from a mismatched load, and the power that reaches the antenna. Click picks a cable, double pins one for comparison. Labeled **approximate**.
SWR	Enter SWR, return loss or reflection coefficient; see all of them plus mismatch loss and the power reflected and delivered.
POWER	W, mW, dBm, dBW and Vrms (across a load you set, 50 ohm by default) with Vpp and an S-meter estimate; dB to power and voltage ratio, and back.
OHM	Ohm's law and power. Enter any two of V, I, R, P: the two most recently edited fields are the inputs.
LC	LC resonance. Solve for frequency, inductance or capacitance; also shows the reactance at resonance.
MATCH	L-network. Series and shunt reactance and the parts for the low-pass or high-pass form.
PATH	Free-space path loss and a simple link budget (power, antenna gains, line loss) to a received level and S-meter estimate.
BANDS	US amateur bands with edges and sub-band notes for the frequency you entered. Click a band row to set the frequency to its centre, double for the lower edge, triple for the upper edge. Labeled **informational only**.

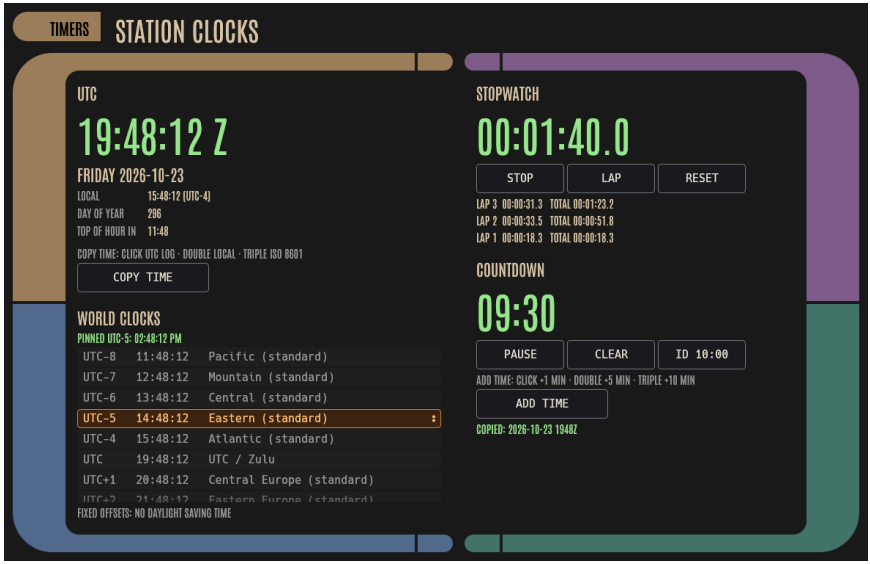
Click modes are used as *variants of one function* here: unit pickers, the direction of a conversion, which quantity to solve for, and the band row's three frequency targets. Nothing destructive sits behind a double or triple click.



Honesty about accuracy. These are bench rules of thumb. Antenna lengths are starting points to trim with an analyzer. Cable loss uses rounded datasheet figures. The band plan and privileges are a summary: check the current FCC rules (Part 97) before transmitting. Each tool says so on screen.

11.4 Timers

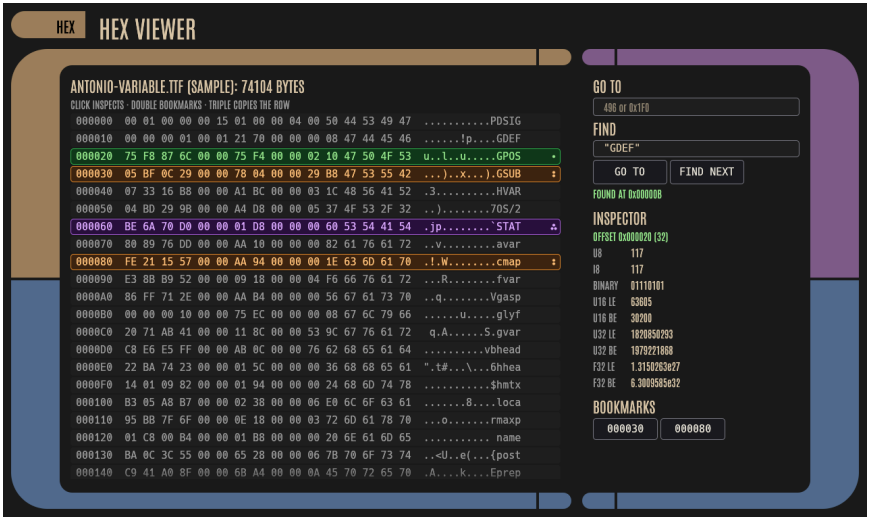
`cargo run --example timers`. The source is in `examples/timers/`: `main.rs` holds the UI and `clock.rs` the stopwatch, countdown, clock formats and zones. `clock.rs` never reads the system clock, because callers pass the time in, so its tests are deterministic.



- **Clocks:** a large UTC clock, local time with its offset, day of year and the time to the top of the hour.
- **COPY TIME** is one momentary `mode_button`: click copies a log-style UTC stamp (`2026-10-23 1948Z`), double the local time with offset, triple an ISO 8601 stamp.
- **World clocks** use `mode_list`. Click pins a zone with a 24-hour face, double a 12-hour face and triple an ISO 8601 face. Only one zone is pinned at a time. The zones are fixed standard offsets with no daylight saving, and the screen says so.
- **Stopwatch** with laps (split and total), and a **countdown** with Start/Pause, Clear and `ID 10:00`, a ten minute timer for station identification (FCC 97.119). ADD TIME is one button: click +1 min, double +5, triple +10.
- When the countdown ends the display turns orange and says `TIME!`, so the state is never shown by color alone.

11.5 Hex viewer

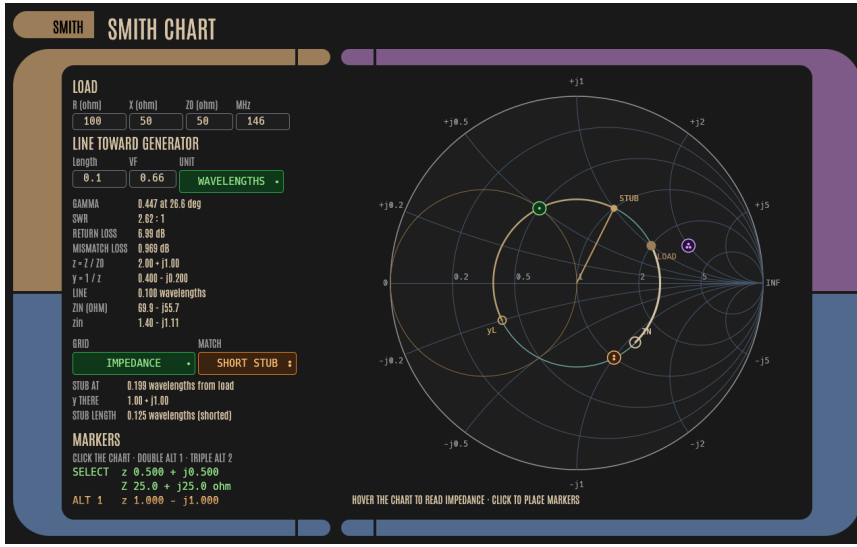
`cargo run --example hexview [-- FILE]`. With no file it shows the bundled Antonio font as a sample. The source is in `examples/hexview/`: `main.rs` holds the UI and `hex.rs` the row formatting, parsing, search and value decoding.



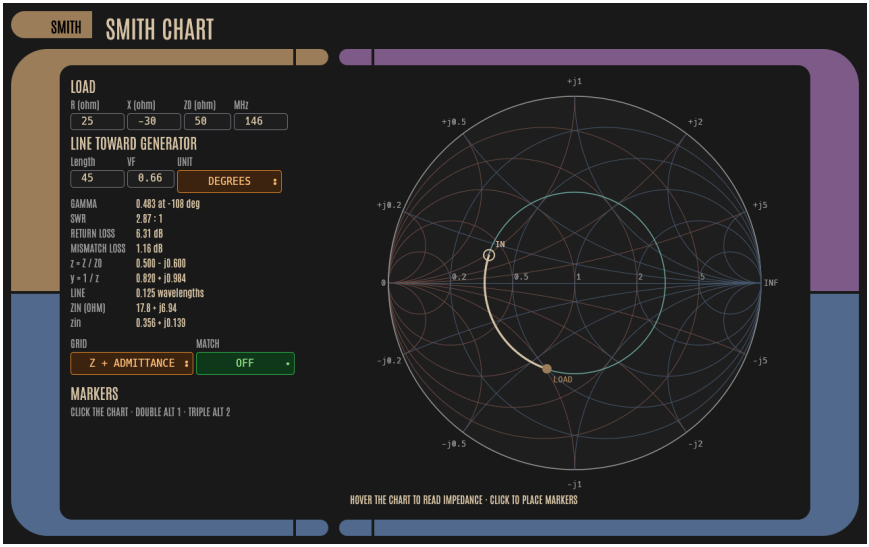
- **Rows** are a virtualised `mode_list` inside `ScrollArea::show_rows`, so a 16 MiB file scrolls smoothly. Files larger than that are cut at 16 MiB and the title says so. Set the list's `item_spacing.y` to the same value `mode_list` uses (2 px), or the scroll offsets drift.
- **Three ways to mark a row:** click inspects it (the INSPECTOR shows U8, I8, binary, U16, U32 and F32 in both byte orders), double-click bookmarks it (bookmarks are listed for a quick jump) and triple-click copies its hex bytes to the clipboard.
- **GO TO** takes `496` or `0x1F0`. **FIND** takes hex bytes (`DE AD BE EF`) or quoted text (`"RIFF"`), searches forward from the selection and wraps once.
- Errors are shown in words (`ERROR: …`) in the status line.

11.6 Smith chart

`cargo run --example smith`. The source is in `examples/smith/`: `main.rs` holds the UI and `math.rs` the complex maths, grid geometry, line rotation and stub matching. The SWR, return-loss and mismatch figures reuse `ham_rf`'s `calc.rs` through a `#[path]` module.



- **Chart:** painted with egui's painter and scaled to the frame. It shows constant-resistance circles and constant-reactance arcs clipped to the unit circle, with labels on the real axis and the rim. The GRID picker adds the admittance grid (the impedance grid mirrored through the centre).
- **Load:** enter R, X, Z0 (50 ohm by default) and the frequency. The load is plotted with its constant-SWR circle. The panel shows Gamma (magnitude and angle), SWR, return loss, mismatch loss and the normalized impedance and admittance.
- **Line:** a length in wavelengths, degrees or metres (with velocity factor) rotates the load clockwise, toward the generator, along the SWR circle. The arc and the input point `IN` are drawn, and Zin is shown in ohms and normalized. A unit test checks the rotation against the textbook formula Z0 (ZL + jZ0 tan bl) / (Z0 + jZL tan bl).
- **Pointer and markers:** hovering reads off z and Z under the pointer. Clicking places markers through `handle_mode` on the chart's own response: click = primary (green), double = alt 1 (orange), triple = alt 2 (purple), each with its pips. *Clear markers* removes them.
- **Match:** the MATCH picker (off, short stub, open stub) computes a single shunt-stub match. It finds the first point toward the generator where the admittance is 1 + jb, then the stub that cancels jb. The path is drawn in the admittance plane: from yL along the SWR circle to the g = 1 circle, then to the centre.



11.7 Patterns worth copying

- **Pickers.** A unit or direction with two or three choices is one `mode_button` showing the current choice. Click selects the first choice, double the second, triple the third. A picker always has a value, so a repeated gesture does not switch it off, and a gesture beyond the number of choices is ignored. A hint line under it names the three gestures. See `pick()` in `ham_rf`. For five or more choices use a row of single-choice buttons (`radio_row()`), not modes.
- **Momentary variants.** Keep a `Mode::Inactive` local, pass it to `mode_button` and act on the result each frame (the calculator's function keys, COPY TIME, ADD TIME).
- **One marked row per meaning.** After `mode_list`, compare the modes with the ones you had, clear the other rows that held the same mode, and then act on the change.
- **Pure core, thin shell.** Put the formulas and parsing in a module with no egui in it and test it with known values. The UI only calls it.

11.8 Screenshots

Each example contains a kittest test that renders it headlessly:

```
cargo test --example calculator   # writes docs/calculator.png
cargo test --example diary        # writes docs/diary.png
cargo test --example ham_rf       # writes docs/ham_rf.png,
```

```
ham_rf_coax.png, ham_rf_swr.png, ham_rf_bands.png
cargo test --example timers       # writes docs/timers.png
cargo test --example hexview      # writes docs/hexview.png
cargo test --example smith        # writes docs/smith.png,
smith_admittance.png
```

`cargo test --example ham_rf` also renders every tool into `target/ham_rf_tools/` so layout problems are easy to spot. `cargo test --example smith` also renders a smaller window to `target/smith_small.png` to check that the chart scales.

12. Checklist and Glossary

12.1 Do

- Take every color, size and gap from `LcarsTheme` or `palette`.
- Change thickness at every elbow, and end each bar with a cap.
- Use `mode_button` / `mode_list` for functions that have related variants, and `lcars_button` for navigation.
- Show modes with color **and** pips, and publish mode names to screen readers.
- Keep text at 4.5:1 or more and non-text UI at 3:1 or more; test it.
- Give each task its own screen, and keep controls in place.
- Keep transitions as fades under 1 s; blink no faster than 1 Hz.
- Keep the web interface off by default, bound to localhost, with devices only behind the bridge.
- Ship `OFL.txt` with the font.

12.2 Don't

- Add gradients, bevels or shadows to frames.
- Use the same thickness on both sides of a turn, or stretch an elbow.
- Add a fourth font size, a sixth color, or decorative uses of the state colors.
- Put destructive actions on double or triple clicks.
- Move or reflow existing controls when something new appears.
- Rely on color alone, or on hover alone.
- Use `std::time::Instant` in shared UI code.

12.3 Glossary

- **Bar:** a straight colored band of the frame. Vertical bars are thick, horizontal bars thin.
- **Cap:** the rounded end of a bar. LCARS's "full stop".

- **Click mode:** one of Inactive / Primary / Alt 1 / Alt 2, chosen by 1–3 clicks or a modifier.
- **Elbow / sweep:** the curved joint between a vertical and a horizontal bar. It is LCARS's signature shape.
- **Frame:** the continuous bent band of bars around content. Here, `LcarsFrame`.
- **Ink:** the label color on a fill, chosen by `ink_on`.
- **OMEGA rule:** the Manifesto's name for the absolute rule that frame thickness changes at every turn.
- **Okudagram:** a nickname for LCARS displays, after designer Michael Okuda.
- **Paint-behind:** reserving a shape slot first and filling it after layout, so chrome can be sized to the content.
- **Pips:** the 1–3 painted dots (·, ⁚, ·:) that show a control's mode.
- **Segment:** a piece of a bar cut out with a gap. It often works as a button.
- **Readout:** a data value display; it is highlighted while being processed.

12.4 Credits

- **Design guidance:** Bracer Jack, *LCARS GuideLine* and *The LCARS Manifesto*, republished by lcars-terminal.de.
- **Structure:** inspired by Apple's Macintosh Human Interface Guidelines (1987/1992). No text was copied.
- **Paint-behind technique:** inspired by paxel/dedup-rs (MIT).
- **Font:** Antonio, © 2013 The Antonio Project Authors, SIL OFL 1.1.

Star Trek and LCARS are trademarks and/or copyrights of their respective owners. rustcars is an unofficial fan project and is not affiliated with or endorsed by them.